

TASK 01 — Database Design & Database-per-Service Architecture

Step 1 — Study database architecture

Relational Database

A **relational database** stores data in tables.

Examples:

- MySQL
- PostgreSQL
- Oracle
- SQL Server

Example, an e-commerce application may have:

Users

Products

Orders

Payments

Each table stores a particular type of information.

Tables

A **table** stores related data in rows and columns.

Example: users

id name email

1 Ravi ravi@gmail.com

2 Kumar kumar@gmail.com

- **Column** → type of information
- **Row** → one complete record

Example, name is a column, while Ravi is a value in a row.

Primary Key

A **primary key** uniquely identifies each record in a table.

Example:

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

Here:

id = Primary Key

The id should be unique.

1 → Ravi

2 → Kumar

3 → Suresh

Two users cannot normally have the same primary-key value.

Foreign Key

A **foreign key** connects one table to another.

Example:

users

id

name

orders

id

user_id

amount

Here:

orders.user_id → users.id

Example:

users

id	name
----	------

1	Ravi
---	------

2	Kumar
---	-------

orders

id	user_id
----	---------

101	1
-----	---

102	2
-----	---

Order 101 belongs to Ravi because:

orders.user_id = 1

users.id = 1

Relationships

Relationships describe **how tables are connected**.

Common types:

One-to-One

One user has one profile.

User — Profile

1 1

One-to-Many

One user can have many orders.

User —< Orders

1 Many

Many-to-Many

Many students can take many courses.

Students >——< Courses

Usually, a separate junction table is used.

Constraints

Constraints are rules that control the data stored in the database.

Common constraints:

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE
- CHECK
- DEFAULT

Example:

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    age INT CHECK (age >= 18),  
    status VARCHAR(20) DEFAULT 'ACTIVE'  
);
```

This prevents invalid data from entering the database.

Indexes

An **index** helps the database find data faster.

Suppose you frequently search users by email:

```
SELECT * FROM users  
WHERE email = 'ravi@gmail.com';
```

You can create an index:

```
CREATE INDEX idx_users_email  
ON users(email);
```

Without a suitable index, the database may need to examine many rows.

With an index, searching can be much faster.

Important: Indexes improve read/search performance but consume storage and can add overhead to INSERT, UPDATE, and DELETE.

Normalization

Normalization means organizing data to reduce duplication and maintain consistency.

Bad design:

order_id	customer_name	customer_phone	product
101	Ravi	9999999999	Laptop
102	Ravi	9999999999	Mouse

Customer information is repeated.

Better design:

Customers

customer_id	name	phone
1	Ravi	9999999999

Orders

order_id	customer_id	product
101	1	Laptop
102	1	Mouse

Now customer information is stored once.

1NF

↓

2NF

↓

3NF

The goal is generally to reduce unnecessary duplication and update anomalies.

Denormalization

Denormalization intentionally introduces some duplication to improve performance or simplify frequently used reads.

Example, instead of repeatedly joining several tables, an application might store a frequently needed value in another table.

Normalization



Less duplication

Better data consistency

Denormalization



Some duplication

Potentially faster/simpler reads

Denormalization should be done carefully because duplicated data can become inconsistent.

In short

Topic	Simple meaning
Database	Stores application data
Table	Organizes data into rows and columns
Primary Key	Uniquely identifies a record
Foreign Key	Connects tables
Relationship	Defines how tables are connected
Constraint	Enforces data rules
Index	Makes searches faster
Normalization	Reduces unnecessary duplication
Denormalization	Adds controlled duplication for performance

Step 2 — Understand database-per-service

User Service → User Database

The **User Service** handles user-related operations.

Example:

Create User

Get User

Update User

Delete User

Login

Manage Profile

Its database contains user information:

User Database

users

id

name

email

phone

password

status

created_at

Only the **User Service** directly manages this database.

User Service



User Database

Order Service → Order Database

The **Order Service** handles orders.

Example:

Create Order

Get Order

Cancel Order

Update Order Status

View Order History

Its database may contain:

Order Database

orders

id

user_id

product_id

quantity

amount

status

created_at

The Order Service owns this data.

Order Service



Order Database

Notice that `user_id` can identify the customer, but the Order Service does **not** directly access the User Database just to get user details.

Payment Service → Payment Database

The **Payment Service** handles payment-related operations.

Example:

Process Payment

Refund Payment

Check Payment Status

Payment History

Its database may contain:

Payment Database

payments

id

order_id

amount

payment_status

transaction_id

created_at

The Payment Service owns this data.

Payment Service

↓

Payment Database

Real-world order example

Suppose **Ravi** places an order.

User exists

User Service

↓

User DB

User:

id = 101

name = Ravi

Ravi creates an order

The client calls:

POST /orders

The Order Service creates:

Order:

id = 5001

user_id = 101

product_id = 20

amount = 5000

The Order Service stores this in:

Order Database

Payment

The Order Service communicates with the Payment Service:

Order Service



Payment Service



Payment Database

Payment information is stored in the Payment Database.

Shared Database

Multiple services use one DB

Strong database coupling

Easier cross-table queries

Easier transactions

Less infrastructure initially

Database changes can affect many services

Database-per-Service

Each service owns its DB

Lower coupling

Cross-service queries are more complex

Distributed transactions are harder

More infrastructure

Services can evolve more independently

Step 3 — Design the User Database

The **User Service** owns the **User Database**.

User Service



User Database



users table

Users table

The table contains these fields:

Field	Purpose
id	Unique identifier for the user
name	User's name
email	User's email address
phone	User's phone number
status	Current user status
created_at	When the user was created
updated_at	When the user was last updated

Id: This uniquely identifies every user.

Example:

id

1

2

3

Usually, this is the **primary key**.

id BIGINT PRIMARY KEY

Example:

1 → Ravi

2 → Kumar

3 → Suresh

Name: Stores the user's name.

name

Ravi

Kumar

Suresh

Example SQL:

name VARCHAR(100) NOT NULL

email: Stores the user's email address.

email

ravi@gmail.com

kumar@gmail.com

Email is often made **unique** so that two accounts cannot use the same email.

email VARCHAR(255) UNIQUE NOT NULL

phone: Stores the user's phone number.

phone

9876543210

9123456780

It can also be made unique if the application requires one account per phone number.

phone VARCHAR(20)

status: Stores the current state of the user.

Example:

ACTIVE

INACTIVE

BLOCKED

Example:

id | name | status

---|-----|-----

1 | Ravi | ACTIVE

2 | Kumar | INACTIVE

3 | Suresh | BLOCKED

created_at

Stores when the user account was created.

Example:

2026-09-28 10:30:00

This is useful for auditing and reporting.

updated_at

Stores when the user's information was last modified.

Example:

2026-09-28 11:15:00

For example, if Ravi changes his phone number, updated_at can be updated.

Complete Users Table

A possible design is:

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  phone VARCHAR(20),  
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Step 4 — Design the Order database

orders Table

The orders table stores information about the overall order.

Example:

orders

order_id

user_id

total_amount

status

created_at

updated_at

Important fields

Field	Purpose
order_id	Unique order identifier
user_id	Identifies the customer
total_amount	Total value of the order
status	Current order status
created_at	When order was created
updated_at	Last update time

Example:

order_id	user_id	total_amount	status
1001	1	5000	CONFIRMED
1002	2	2500	PENDING

order_items Table

An order can contain multiple products.

Order 1001

- └─ Laptop × 1
- └─ Mouse × 2
- └─ Keyboard × 1

Instead of putting all products into one row, we create an order_items table.

Example:

order_items

order_item_id

order_id

product_id

quantity

unit_price

Fields

Field	Purpose
order_item_id	Unique item-record ID
order_id	Identifies the order
product_id	Identifies the product
quantity	Number of products
unit_price	Price of one product

Example:

order_item_id	order_id	product_id	quantity	unit_price
1	1001	501	1	4000
2	1001	502	2	500
3	1001	503	1	1000

So order 1001 contains three different items.

Relationship Between orders and order_items

The relationship is:

orders

| 1

| _____ <

| | Many

order_items order_items

More simply:

One Order

↓

Many Order Items

Example:

Order 1001

└─ Item 1 → Laptop
└─ Item 2 → Mouse
└─ Item 3 → Keyboard

Therefore:

orders.order_id

↓

order_items.order_id

order_items.order_id is a **foreign key** referencing orders.order_id.

Complete Order Database Design

orders

```
CREATE TABLE orders (  
    order_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    user_id BIGINT NOT NULL,  
    total_amount DECIMAL(12,2) NOT NULL,  
    status VARCHAR(30) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

order_items

```
CREATE TABLE order_items (  
    order_item_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    order_id BIGINT NOT NULL,  
    product_id BIGINT NOT NULL,  
    quantity INT NOT NULL,  
    unit_price DECIMAL(12,2) NOT NULL,  
    FOREIGN KEY (order_id)  
        REFERENCES orders(order_id)  
);
```

Step 5 — Design the Payment Database

The **Payment Service** owns the Payment Database.

Payment Service



Payment Database



payments table

id

This is the unique identifier for each payment.

Example:

id

1

2

3

Usually: id BIGINT PRIMARY KEY

Example:

Payment 1 → Order 1001

Payment 2 → Order 1002

order_id

This identifies the order for which the payment was made.

Example:

order_id = 1001

means:

Payment → Order 1001

Important microservices point

Because this is **Database-per-Service**, don't create a database-level foreign key from Payment DB to Order DB.

Instead:

Payment Service

| order_id = 1001

Order Service

The Payment Service can communicate with the Order Service through an API or event when it needs information about the order.

amount

Stores the amount being paid.

Example:

amount = 5000.00

A suitable database type is:

DECIMAL(12,2)

This is preferable to floating-point types for monetary values.

Example:

Order Amount = ₹5,000

Payment Amount = ₹5,000

payment_status

Stores the current state of the payment.

Typical values might be:

PENDING

SUCCESS

FAILED

REFUNDED

Example:

id | order_id | amount | payment_status

---|-----|-----|-----

1 | 1001 | 5000 | SUCCESS

2 | 1002 | 2500 | PENDING

3 | 1003 | 1500 | FAILED

The exact statuses depend on your application's business rules.

transaction_reference

This stores the reference/identifier returned by the payment provider.

Example:

transaction_reference

TXN_ABC123456

TXN_XYZ987654

This is useful when you need to investigate or reconcile a payment with the payment provider.

created_at

Stores when the payment record was created.

Example:

2026-09-28 10:30:00

This helps with:

- Payment history
- Auditing
- Reports
- Troubleshooting
- **Payment Table**

A possible design is:

```
CREATE TABLE payments (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  payment_status VARCHAR(30) NOT NULL,
  transaction_reference VARCHAR(255) UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Notice that we don't use:

```
FOREIGN KEY (order_id) REFERENCES orders(order_id)
```

because orders belongs to the **Order Database**, not the Payment Database.

Step 6 — Create ER Diagrams

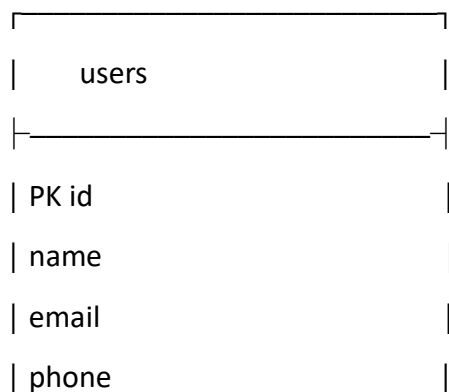
What is an ER diagram:

ER = Entity Relationship

It visually represents:

- Tables/entities
- Columns/attributes
- Primary keys
- Foreign keys
- Relationships

User DB ER Diagram

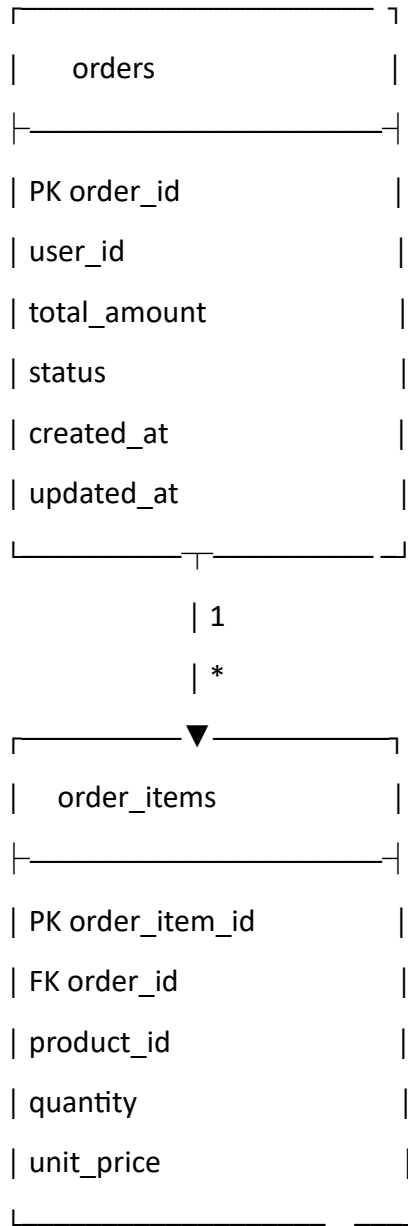


status	
created_at	
updated_at	

This diagram belongs to the **User Service**.

Order DB ER Diagram

The Order Database has two tables:



The relationship is:

One Order → Many Order Items

Payment DB ER Diagram

The Payment Database has:

payments	
PK id	
order_id	
amount	
payment_status	
transaction_reference	
created_at	

Here, order_id identifies the related order at the **application/service level**, rather than being a cross-database foreign key.

Step 7 — Define Ownership

User Service owns users

User Service



User Database



users

The User Service is responsible for:

Create user

Update user

Get user

Delete/deactivate user

Manage user status

Other services should not directly modify the users table.

Order Service owns orders

Order Service



Order Database



orders

order_items

The Order Service manages:

Create order

Update order

Cancel order

Get order

Manage order status

The User Service should not directly modify the orders table.

Payment Service owns payments

Payment Service



Payment Database



payments

The Payment Service manages:

Create payment

Process payment

Update payment status

Track transaction reference

Process refunds

The Order Service should not directly modify the payments table.

Complete Order + Payment Flow

Suppose Ravi places an order for ₹5,000.

Step 1 — User

User Service



User DB

User ID = 101

Name = Ravi

Step 2 — Order

Order Service



Order DB

Order ID = 1001

User ID = 101

Amount = ₹5,000

Status = CREATED

Step 3 — Payment

Order Service



Payment Service



Payment DB

Order ID = 1001

Amount = ₹5,000

Status = SUCCESS

Transaction = TXN12345

Step 4 — Result

The Payment Service can communicate the payment result back to the Order Service.

Payment Service



Payment successful



Order Service



Order status updated

For larger microservices systems, this communication may use an event/message broker such as Kafka.

Step 8 — Remove Direct Database Dependency

Wrong approach

Suppose the Order Service needs the user's name.

It should **not** do this:

Order Service



User Database ❌

Example, the Order Service should not connect directly to the User Database and execute:

```
SELECT name
```

```
FROM users
```

```
WHERE id = 101;
```

Why:

Because the users table belongs to the **User Service**.

The User Service owns and controls that data.

Step 9 — Define Service Communication

Suppose we have:

User ID = 101

The Order Service needs to know the user's details.

Order Service sends request

GET /users/101

The request goes to the User Service.

Order Service

| GET /users/101

User API

User Service receives request

The User Service processes the request.

User API



User Service

User Service queries its own database

User Service



User Database



users table

It may execute something like:

SELECT id, name, email, phone

FROM users

WHERE id = 101;

User Service returns the response

For example:

```
{  
  "id": 101,  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

The response goes back:

User Database



User Service



User API



Order Service

Service-to-service communication should happen through APIs or messaging, not direct database access.

Step 10 — Document Architecture

Database Overview

First, explain the overall architecture.

For this project:

User Service



User Database

Order Service



Order Database

Payment Service



Payment Database

Explanation

Each microservice has its own database.

This follows the **Database-per-Service pattern**.

Database Schemas

Document the tables and columns for each database.

User Database

users

id

name

email

phone

status

created_at

updated_at

Order Database

orders

order_id

user_id

total_amount

status

created_at

updated_at

order_items

order_item_id

order_id

product_id

quantity

unit_price

Payment Database

payments

id

order_id

amount

payment_status

transaction_reference

created_at

ER Diagrams

Include a separate ER diagram for each database.

User DB

users		

PK id		
name		
email		
phone		
status		
created_at		
updated_at		

Order DB

orders		

PK order_id		

user_id	
total_amount	
status	
└──────────┘	

| 1 : Many

order_items	
└──────────┘	

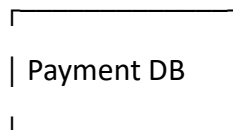
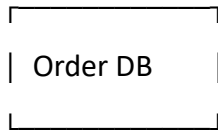
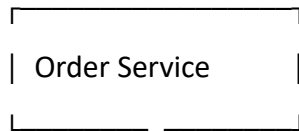
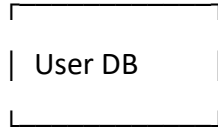
PK item_id	
order_id	
product_id	
quantity	
unit_price	
└──────────┘	

Payment DB

payments	
└──────────┘	
PK id	
order_id	
amount	
payment_status	
transaction_reference	
created_at	
└──────────┘	

Database-per-Service Architecture

Show the complete architecture:



This diagram makes database ownership clear.

Data Ownership

Create a table like this:

Service	Database	Owns
User Service	User DB	User data
Order Service	Order DB	Order data
Payment Service	Payment DB	Payment data

Important rule

User Service → owns users

Order Service → owns orders

Payment Service → owns payments

No service should directly modify another service's database.

Service Communication

Document how services communicate.

Wrong

Order Service



User Database ❌

Correct

Order Service



User API



User Service



User Database

For example:

GET /users/101

The User Service retrieves the user from its own database and returns the response.

For asynchronous communication, services can also communicate through events/messages.

Order Service



OrderCreated Event



Message Broker



Payment Service

SQL Scripts / Migrations

Your project should contain database creation scripts or migration files.

For example:

database/

├─ user-db/

└─ V1__create_users.sql

├─ order-db/

└─ V1__create_orders.sql

└─ V2__create_order_items.sql

└─ payment-db/

└─ V1__create_payments.sql

A migration represents a controlled database change.

Example:

V1 → Create users table

V2 → Add status column

V3 → Add email index

This allows developers to reproduce the same database structure in DEV, TEST, and PROD.

Design Decisions

This is an important part of Step 10.

Explain **why** you made the design choices.

For example:

Decision 1 — Database per Service

Reason: Each microservice should independently own and manage its data.

Decision 2 — No cross-database foreign keys

Reason: Databases belong to different services. Relationships across services are handled through APIs or messaging.

Decision 3 — orders and order_items are separate

Reason: One order can contain multiple products.

Decision 4 — Index frequently searched fields

Reason: Indexes can improve query performance for common lookups.

Decision 5 — Store timestamps

Reason: created_at and updated_at help with auditing and troubleshooting.